VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothri Kalan, Sehore

Madhya Pradesh - 466114

Project Report

Driver Drowsiness Detection System

Submitted for: Computer Vision Course Assignment
B.Tech — Computer Science & Engineering(AIML)

Academic Year: 2024–25

SUBMITTED BY- PRISHA
REGISTRATION NO-23BAI10881
SUBMITTED TO-DR. AMRITA PARASHAR
SLOT-F11+F12

1. Problem Statement

Every year, hundreds of thousands of road accidents are caused by driver drowsiness and fatigue. Unlike distracted driving or speeding, drowsiness is difficult for drivers to self-monitor — by the time a person realises they are falling asleep, it is often too late. Studies by the National Highway Traffic Safety Administration (NHTSA) estimate that drowsy driving contributes to approximately 100,000 reported crashes annually in the United States alone. In India, the problem is equally severe, with long-distance truck and bus drivers being especially at risk due to extended driving hours.

The core challenge is detection: how do you reliably identify that a driver is becoming drowsy before an accident occurs, using hardware that is already available in most vehicles or smartphones?

Existing solutions fall into two broad categories. Physiological methods such as EEG (electroencephalography) and EOG (electrooculography) are highly accurate but require invasive sensors attached to the driver's body, making them impractical for everyday use. Vehicle-based methods monitor steering wheel patterns or lane deviation, but by the time these signals appear, the driver is already dangerously impaired.

This project addresses the problem with a third approach: non-intrusive vision-based detection using a standard webcam. By analysing the driver's face in real time, the system can detect eye closure patterns that are strong predictors of drowsiness, and issue an audio alert before the driver loses control.

2. Why This Problem Matters

The motivation for choosing this project goes beyond the technical challenge. It is a problem with direct, measurable real-world impact:

- Human lives are at stake. Drowsy driving is as dangerous as drunk driving, yet far less stigmatised and far less monitored.
- The solution is accessible. Unlike EEG-based systems that cost thousands of dollars, a webcam-based system can be deployed on any laptop, smartphone, or a low-cost Raspberry Pi — making it viable for public transport and freight vehicles in developing countries.
- Computer vision is well-suited to this problem. The human eye exhibits measurable geometric changes as it closes. These changes can be tracked reliably using facial landmark detection, making this a strong application of the techniques studied in this course.
- It is deployable. The system runs in real time on standard hardware with no internet connection, making it practical for actual field use.

Choosing a problem with genuine societal relevance made every technical decision feel purposeful — whether it was tuning the EAR threshold or deciding how many frames to wait before triggering the alarm.

3. Approach

3.1 Choosing the Detection Method

After researching available techniques, I chose the Eye Aspect Ratio (EAR) method, originally proposed by Soukupova and Cech in their 2016 paper on real-time eye blink detection. The EAR approach is well-documented, computationally lightweight, and has been validated in published research.

The central insight is geometric: the human eye can be described by six landmark points. The ratio of vertical distances to horizontal distance stays approximately constant when the eye is open (~0.30) and drops sharply toward zero when the eye closes. Drowsiness is characterised not by a single blink but by prolonged or repeated eye closure — which the EAR method captures by checking whether the ratio stays below a threshold for a sustained number of frames.

```
EAR = ( ||p2 - p6|| + ||p3 - p5|| ) / ( 2 × ||p1 - p4|| )
```

p1 through p6 are the six facial landmark coordinates around a single eye, computed using dlib's 68-point shape predictor.

3.2 Technology Stack

The following libraries were selected based on maturity, documentation quality, and suitability for real-time processing:

Library	Version	Role
Python	3.x	Primary language — readable, strong CV ecosystem
OpenCV (cv2)	4.x	Webcam capture, frame processing, visual overlay
dlib	20.x	HOG-based face detection + 68-point landmark prediction
imutils	0.5.4	Convenience utilities for resizing and landmark indexing
scipy	1.17	Euclidean distance calculations for EAR formula
pygame	2.6	Cross-platform audio alarm playback

3.3 System Pipeline

The detection pipeline processes each webcam frame through five sequential stages:

Stage	Operation	Tool
1	Capture frame from webcam	cv2.VideoCapture
2	Convert to grayscale for faster processing	cv2.cvtColor
3	Detect face bounding box in the frame	dlib frontal_face_detector
4	Map 68 facial landmarks onto the detected face	dlib shape_predictor
5	Extract eye points, compute EAR, check threshold	scipy + custom logic
6	Trigger alarm and display alert if threshold exceeded	pygame + cv2.putText

4. Key Decisions

4.1 EAR Threshold: 0.25

The EAR threshold determines what eye openness level counts as 'closed.' Too high, and normal blinking triggers false alarms. Too low, and drowsy eyes are missed. Through live testing on my own face under various lighting conditions, I found that 0.25 provided the best balance — consistent with the value recommended in Soukupova and Cech's original paper. This value correctly distinguishes a fast blink (EAR dips briefly then recovers) from sustained eye closure.

4.2 Frame Counter: 20 Frames

Rather than alarming on a single low-EAR frame (which would fire on every blink), the system requires the EAR to stay below the threshold for 20 consecutive frames. At a typical webcam rate of 30fps, this corresponds to approximately 0.67 seconds — long enough to filter out normal blinks, short enough to catch early drowsiness. This design choice significantly reduced false positives during testing.

4.3 dlib over OpenCV's Haar Cascades

OpenCV provides built-in Haar cascade classifiers for eye detection, which are faster but less accurate. dlib's HOG-based face detector combined with the 68-point landmark predictor gives more stable and precise eye coordinates, especially under slight head rotation or imperfect lighting. Since the entire system depends on accurate eye landmark positions, I prioritised accuracy over raw speed.

4.4 Averaging Both Eyes

EAR is computed separately for the left and right eye, then averaged. This guards against cases where one eye is partially occluded (e.g., by glasses frames, shadows, or a slight head turn), which could otherwise cause a false trigger. Averaging makes the detection more robust.

4.5 dlib-bin instead of building dlib from source

During setup, building dlib from source on Windows failed due to a CMake path conflict — a known issue documented in dlib's own error messages. Rather than spending time debugging build toolchains, I switched to the dlib-bin package, which provides a pre-compiled wheel for Windows. This was the right engineering trade-off: it unblocked progress immediately and did not affect functionality.

5. Challenges Faced

5.1 dlib Installation on Windows

The first major obstacle was getting dlib to install on Windows with Python 3.12. The standard pip install dlib fails because dlib must be compiled from C++ source code, which requires CMake to be installed and correctly added to the system PATH. The pip install cmake command installs a Python-managed cmake binary that dlib's build system does not recognise as valid.

Resolution: Using pip install dlib-bin, which provides pre-compiled binaries for Windows, resolved the issue completely without any build configuration. This experience highlighted the importance of knowing the difference between a source distribution and a binary wheel in Python packaging.

5.2 Calibrating Sensitivity

Early versions of the system either fired the alarm too frequently (even during normal blinking) or were too slow to respond. The root cause was using a threshold and frame count that were not calibrated to actual usage. Testing by deliberately blinking slowly, keeping eyes half-open, and simulating drooping eyelids helped converge on the final values of EAR = 0.25 and FRAME_LIMIT = 20.

5.3 Performance in Varied Lighting

dlib's face detector performs well under standard indoor lighting but struggles in dim conditions or when the face is strongly backlit. During testing, I found that placing a light source in front of the face (rather than behind) significantly improved detection reliability. For a production system, adding adaptive histogram equalisation (cv2.createCLAHE) to the preprocessing step would help, though this was not implemented in the current version.

5.4 Glasses and Occlusion

Thick-framed glasses partially occlude the eye region and can cause EAR to read artificially low even when eyes are open. This was observed during testing. The two-eye averaging partially mitigates this, but a robust solution would require training or fine-tuning the landmark predictor on a dataset with glasses — beyond the scope of this project.

6. Results

The system was tested under multiple conditions to evaluate detection accuracy and alarm timing:

Test Scenario	Expected Behaviour	Observed Result
Normal driving (eyes open)	No alarm, green eye contours visible	Pass — EAR steady at ~0.30
Single blink	No alarm triggered	Pass — counter resets before FRAME_LIMIT
Eyes closed for 2 seconds	Alarm triggers	Pass — alarm fired at ~0.67s consistently
Looking sideways (head turned ~15 deg)	Detection continues	Pass — minor EAR fluctuation, no false alarm
Low light environment	Detection may degrade	Partial — face detection slower, EAR less stable
Wearing glasses	Detection works with minor EAR reduction	Pass with thin frames; unreliable with thick frames

7. What I Learned

7.1 Computer Vision is as much about data as algorithms

The EAR algorithm is simple in theory — it is just a ratio of distances. What makes it work in practice is having reliable, accurate facial landmarks to feed into it. I learned that the quality of the upstream detection (face detection and landmark prediction) determines the ceiling of what any downstream algorithm can achieve. Garbage in, garbage out applies directly here.

7.2 Real-time constraints change design decisions

When building a batch image classifier, you can afford to use heavyweight models. A real-time system that must process 30 frames per second demands different choices. This project taught me to think about processing time per frame, not just accuracy. dlib's HOG detector is less accurate than a deep CNN detector but fast enough for real-time use on a CPU — that trade-off was the right one for this application.

7.3 Threshold tuning requires empirical testing

I initially expected to find a single correct threshold value in the literature and use it. In practice, the right threshold depends on the specific camera, lighting, and face geometry. Testing on a real webcam rather than a static dataset taught me that deployed computer vision systems often need per-environment calibration, and that building in easy configurability (as constants at the top of the file) is good engineering practice.

7.4 Error handling and installation matter

The dlib build failure was a lesson in the gap between a working prototype and a reliably installable system. A project that works on one machine but fails to install on another has limited practical value. Documenting dependencies carefully and using binary packages where possible makes a real difference for usability.

7.5 Simple baselines often work

I initially considered using a deep learning-based approach with a CNN trained on eye-state datasets. After researching the EAR method, I realised that a geometric approach based on six landmark points performs comparably for this specific task, with far less complexity, no GPU requirement, and no training data. This reinforced the principle of starting with the simplest approach that could work before adding complexity.

8. Future Improvements

- Yawn detection: Add mouth aspect ratio (MAR) analysis to detect yawning as a secondary drowsiness indicator.
- Head pose estimation: Detect forward head nodding, which is another strong drowsiness signal.
- Adaptive thresholding: Calibrate EAR baseline per user at session start to handle individual facial variation.
- Low-light enhancement: Apply CLAHE (Contrast Limited Adaptive Histogram Equalisation) to improve detection in dim environments.
- Embedded deployment: Port to Raspberry Pi with a camera module for automotive integration.
- Logging and analytics: Record drowsiness events with timestamps for post-trip fatigue analysis.

9. References

[1] Soukupova, T. & Cech, J. (2016). Real-Time Eye Blink Detection using Facial Landmarks. 21st Computer Vision Winter Workshop, Rimske Toplice, Slovenia.

[2] Kazemi, V. & Sullivan, J. (2014). One Millisecond Face Alignment with an Ensemble of Regression Trees. Proceedings of the IEEE CVPR 2014.

[3] National Highway Traffic Safety Administration (NHTSA). Drowsy Driving. https://www.nhtsa.gov/risky-driving/drowsy-driving

[4] OpenCV Documentation. https://docs.opencv.org

[5] dlib C++ Library. http://dlib.net

[6] Rosebrock, A. (2017). Drowsiness Detection with OpenCV. PyImageSearch. https://pyimagesearch.com

[7] Ramzan, M., Khan, H.U., Awan, S.M., Ismail, A., Ilyas, M. and Mahmood, A., 2019. A survey on state-of-the-art drowsiness detection techniques. *IEEE Access*, *7*, pp.61904-61919. https://ieeexplore.ieee.org/iel7/6287639/8600701/08704263.pdf

[8] Fu, B., Boutros, F., Lin, C.T. and Damer, N., 2024. A survey on drowsiness detection–modern applications and methods. *IEEE Transactions on Intelligent Vehicles*. https://arxiv.org/pdf/2408.12990?

[9] Deng, W. and Wu, R., 2019. Real-time driver-drowsiness detection system using facial features. *Ieee Access*, *7*, pp.118727-118738. https://ieeexplore.ieee.org/iel7/6287639/8600701/08808931.pdf

[10] Ngxande, M., Tapamo, J.R. and Burke, M., 2017. Driver drowsiness detection using behavioral measures and machine learning techniques: A review of state-of-art techniques. *2017 pattern recognition Association of South Africa and Robotics and mechatronics (PRASA-RobMech)*, pp.156-161. https://www.academia.edu/download/107134125/Ngxande_20148_2017.pdf